OpenClaw Arm Fixed-point Placement

OpenClaw Arm Fixed-point Placement

1. Course Content
    Course Overview
2. Preparation
3. Arm Fixed-point Placement Case (Depth Camera)
4. Fixed Action Placement (USB Monocular Camera)
5. Source Code Analysis
    5.1 `GetPlacePoint()` — Obtain the placement point coordinates (depth camera)
    5.2 `place()` — Fixed-point placement of an item (depth camera)
    5.3 `PutDown` Fixed Action Group Placement (Monocular Camera)
6. Parameter Debugging
7. Function Q&A
    7.1 How to modify the distance between the car and the target point when placing an object?
    7.2 How to adjust when the placement point is inaccurate?

1. Course Content

Course Overview

- **The robotic arm placement function of the USB monocular camera is implemented through fixed action groups**. Only the depth camera version can perform fixed-point placement based on semantics.
- In daily life and industrial sorting, we rarely throw objects around at will. Instead, we often give commands with clear spatial semantics:

  > *"Put the yellow block you just picked up right between the red block and the blue block."*
  > *"Put the toolbox in the empty spot at the upper-right corner of the table."*

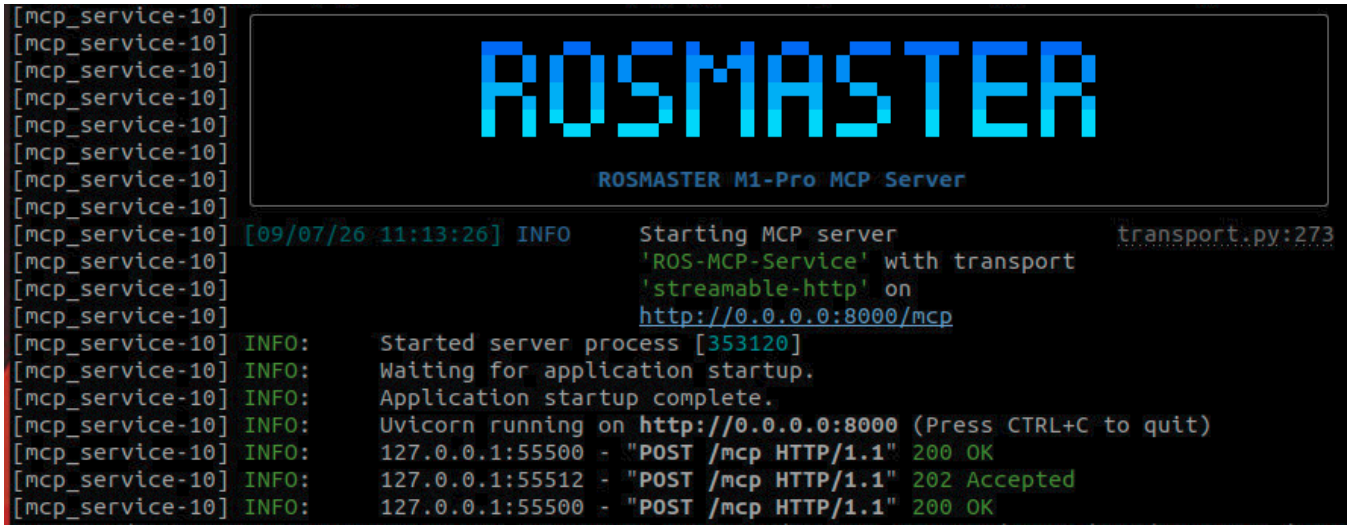
  This is **semantic fixed-point placement**:

  1. Understand spatial relations (semantic perception): The multimodal large model (VLM) needs to understand natural language spatial descriptions such as "in the middle", "on the left", and "empty spot", and find the most suitable target point from the camera frame.
  2. Calculate spatial coordinates (3D back-projection): Reverse-solve the target pixel point on the 2D image into the robot's 3D spatial coordinates (X, Y, Z) through RGB-D depth.
  3. Reachability and inverse kinematics solution (IK solution): Check whether the point is within the maximum arm span of the robotic arm; if it is within range, calculate the angles of each joint to deliver the block steadily and release the gripper gently.

2. Preparation

- Start the MCP service. If you need voice replies, you must add the use_tts parameter: `use_tts:=true` and start the ASR node separately, otherwise it can be omitted.

```
ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true
```

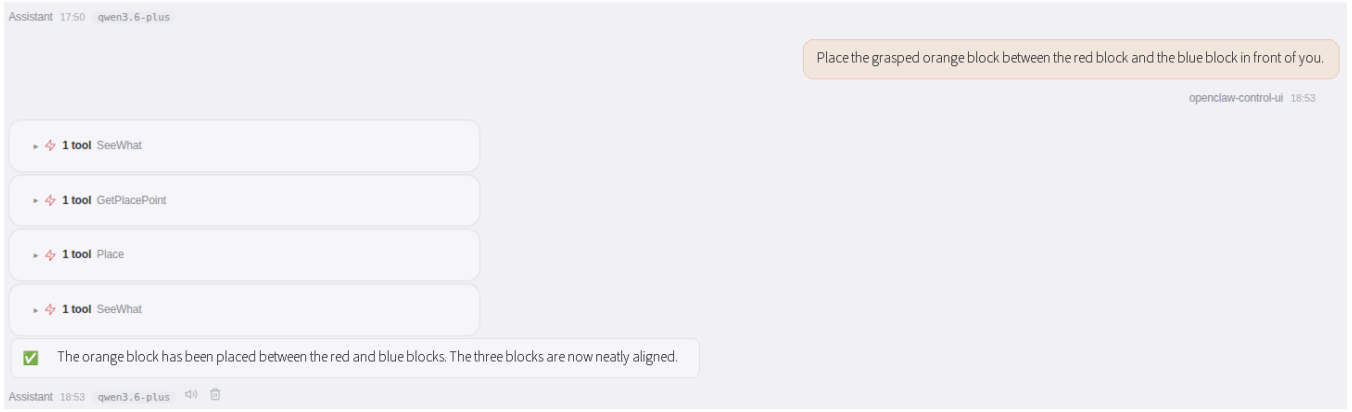


3. Arm Fixed-point Placement Case (Depth Camera)

> **Tip**
>
> - You can write the command yourself according to your needs. The following is a case demonstration. Just choose any one of the interaction methods. The demonstration here uses the web-based WebChat as an example.

Command	Parameter	Function
GetPlacePoint	`--query`	Find the placement point coordinates and check whether they are within the reachable workspace of the robotic arm
Place	`--place-x`, `--place-y`, `--place-z`	Place the object at the specified coordinate position

- The place tool provides the function of placing an object at a specified position in the robot's field of view according to a natural language description. For example: put the orange block you grasped between the red block and the blue block in front of you.



- When obtaining the placement point, a point verification image is automatically generated to facilitate

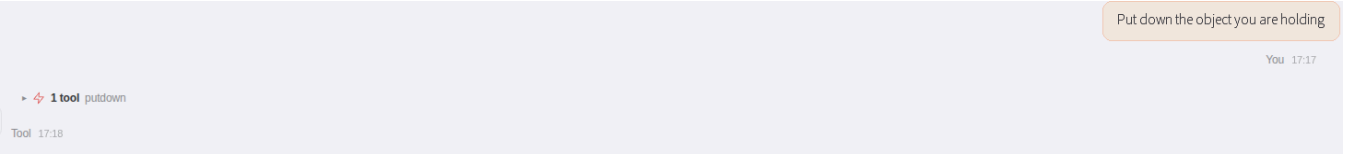
debugging and verification of whether the placement point is accurate. Image path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/cache/image_point_selected.png
```



4. Fixed Action Placement (USB Monocular Camera)

- Put down the object in your hand: The USB camera cannot obtain depth information from the environment, so putting down the object is implemented through fixed actions.



5. Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

5.1 `GetPlacePoint()` — Obtain the placement point coordinates (depth camera)

Process Description:

1. First capture the current frame and overlay a 20px grid reference line (to facilitate visual model positioning).
2. Call the vision_llm vision model to parse the natural language instruction and return the pixel coordinates (x,y) of the placement position.
3. If it returns `(-1, -1)`, it means the target is not in the frame, and an error is returned in advance.
4. Draw a point verification image to `place_verify_image.png`, with a black circle marking the range and a red dot marking the center, making it easy to debug.
5. Convert the pixel coordinates to 3D spatial coordinates through the `get_target_pose` service.
6. Use `place_distance[2]` (x upper limit) and `place_distance[3]` (y upper limit) to determine whether the target point is within the working range of the robotic arm.

```
    def GetPlacePoint(self,query:str)->list[bool,str]:
        save_result=self.SeeWhat()
        if  not save_result[0]:
            return [False, f"SeeWhat error:{save_result[1]}",""]
        try:
            tool = self._get_place_point_tool()
        except Exception as e:
            self.get_logger().error(f"GetPlacePoint init failed: {e}")
            return [False, f"GetPlacePoint failed: {e}", ""]
        # Grid overlay and target-point annotation are handled inside FindTargetPoint
        result = tool.FindTargetPoint(image_path=self.image_path,
target_description=str(query))
        if result.error:
            self.get_logger().error(result.error_message)
            return [False, f"GetPlacePoint failed: {result.error_message}", ""]
        if not result.found:
            return [False, "point is empty; the target point may not exist in the frame.",
""]
        x, y = result.x, result.y

        request = GetTargetPose.Request(x=float(x), y=float(y), radius=self.target_circle)
        result:GetTargetPose.Response = self.get_pose_client.call(request)

        if not result.success:
            self.get_logger().error(f"GetTargetPose failed: {result.message}")
            return [False, str(result.message),""]

        point = str({"x": round(result.pose.position.x, 2),
                     "y": round(result.pose.position.y, 2),
                     "z": round(result.pose.position.z, 2)
                     })
        self.get_logger().info(
                f'Target Point:x:{result.pose.position.x:.2f}, '
```

```
                f'y:{result.pose.position.y:.2f}, '
                f'z:{result.pose.position.z:.2f}')
        if result.pose.position.x<=self.place_distance[2] and result.pose.position.y <=
self.place_distance[3]:
            msg="✅ The target point is within the working range of the robotic arm"
        else:
            msg="❌ The target point is outside the working range of the robotic arm"
        return [True,msg,point]

    def _get_place_point_tool(self):
        """Lazy load the placement-point positioning tool"""
        if self._place_point_tool is None:
            from robot_util.visual_positioning.visual_positioning import
RobotVisualPositioning
            from robot_util.robot_tool import GetRobotInfo
            vision_llm = GetRobotInfo().get("vision_LLM") or {}
            api_key = vision_llm.get("api_key")
            base_url = vision_llm.get("base_url")
            model = vision_llm.get("model")
            if not api_key or not base_url or not model:
                raise RuntimeError("vision_LLM api_key/base_url/model are not configured in
robot_info.json")
            self._place_point_tool = RobotVisualPositioning(api_key=api_key,
base_url=base_url, model=model)
        return self._place_point_tool
```

5.2 `place()` — Fixed-point placement of an item (depth camera)

1. **Offset superposition**: Superimpose the x/y/z offset of `place_offset` on the basis of the target placement coordinates, to fine-tune the placement position.
2. **Inverse kinematics solution**: Call the `/get_kinemarics` service (kin_name="ik") to solve the 6 joint angles according to the target pose.
3. **Joint control**: Publish the 6 joint angles to the `arm6_joints` topic through `_pubSix_Arm()` to drive the robotic arm to move to the placement position.
4. **Release the object**: Control the 6th joint (the gripper) to open (joint=0) to release the grasped object.
5. **Pose recovery**: After placement is complete, call `_pubSix_Arm(self.init_arm_pose)` to restore the robotic arm to its initial pose, avoiding interference with subsequent operations.

```
def place(self, place_x: float, place_y: float, place_z: float) -> list[bool, str]:
    """The robotic arm places the object to the target position."""
    # Step 1: execute the placement action
    self._place(place_x, place_y, place_z, place_pitch=self.place_offset[3])

    # Step 2: automatic retreat
    self.move_by_odom(direction='backward', distance=self.auto_back_dist)
    self.get_logger().info("Place object complete")
    return [True, ""]
def _place(self, place_x, place_y, place_z,
```

```
            place_roll=0.0, place_pitch=0.0, place_yaw=0.0):
    # Step 1: build the inverse kinematics request and superpose the placement offset
    request = ArmKinemarics.Request()
    request.tar_x = place_x + self.place_offset[0]
    request.tar_y = -(place_y + self.place_offset[1])  # Negate the Y axis (coordinate
system conversion)
    request.tar_z = place_z + self.place_offset[2]
    request.kin_name = "ik"
    request.roll = place_roll
    request.pitch = place_pitch
    request.yaw = place_yaw

    # Step 2: call the inverse kinematics solver
    response: ArmKinemarics.Response = self.arm_kin_client.call(request)
    self.get_logger().info(f"place IK-pose: {response}")

    # Step 3: extract the joint angles and control the robotic arm to move
    joints = [0, 0, 0, 0, 0, 0]
    joints[0] = int(response.joint1)
    joints[1] = int(response.joint2)
    joints[2] = int(response.joint3)
    joints[3] = int(response.joint4)
    joints[4] = int(response.joint5)
    joints[5] = self.GraspKeepPose[5]  # The 6th joint keeps the grasping pose angle
    self._pubSix_Arm(joints)
    time.sleep(4.0)  # Wait for the robotic arm to reach the target position

    # Step 4: open the gripper
    self.SingleJoint_pub.publish(ArmJoint(id=6, joint=0, time=1500))
    time.sleep(2.0)  # Wait for the object to be released

    # Step 5: restore the robotic arm to its initial pose
    self._pubSix_Arm(self.init_arm_pose)
```

5.3 `PutDown` Fixed Action Group Placement (Monocular Camera)

```
    def PutDown(self) -> list[bool, str]:
        '''Put down the currently grasped object: move to the putdown pose, open the gripper
to release, then return to the initial pose. Only available for usb camera.'''
        if (guard := self._check_camera_action("PutDown", "usb")) is not None:
            return guard
        self._pubSix_Arm(list(self.putdown_joints[:5]) + [self.GraspKeepPose[5]])
        time.sleep(2.5)
        self.SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1000))
        time.sleep(1.0)
        self._pubSix_Arm(self.init_arm_pose)
        return [True, ""]
```

6. Parameter Debugging

- Parameter file path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

The following is the mapping table between parameters and code:

Parameter	Location used in code	Function description
`place_distance`	`GetPlacePoint()` → `self.place_distance[2]/[3]`	The placement position xyz coordinates, and the tolerance detection of the robotic arm working range (`[2]` is the x upper limit, `[3]` is the y upper limit)
`place_offset`	`_place()` → `self.place_offset[0]/[1]/[2]/[3]`	Placement position offset, used to fine-tune the placement coordinates
`auto_back_dist`	`pick()`、`place()` → `self.auto_back_dist`	Automatic retreat distance after grasping/placement is complete, to avoid collisions and entering the costmap restricted area
`target_circle`	`GetPlacePoint()` / `_get_pick_target_pose()` → `self.target_circle`	When obtaining the target point coordinates, the sampling radius (in pixels) of the circular area used to compute the 3D pose

```
mcp_service:
  ros__parameters:
    #action_controller--Arm
    GraspKeepPose: [90,125,3,0,90,140]          #grasping hold pose of the robotic arm
    grasp_distance: [0.265, 0.0, 0.0]           # [x, y, yaw]
    place_distance: [0.39, 0.0,0.45, 0.25]      #placement position and tolerance detection
    grasp_offset: [0.015,0.015,-0.03,2.0]       # grasp offset x,y,z, grasp angle
    place_offset: [-0.01,0.00,0.05,1.3]         #placement position offset
    auto_back_dist: 0.13                        # Automatic retreat distance, in meters;
after grasping and placement are complete, automatically retreat a distance to avoid
collisions and entering the costmap restricted area

    target_circle : 10                        # When obtaining the target point coordinates,
the circular area radius, in pixels
```

Parameter Description:

- `GraspKeepPose`: The grasping hold pose of the robotic arm, in the format `[J1, J2, J3, J4, J5, J6]`, unit: degrees. J6 is the gripper opening/closing angle (0-180); the larger the value, the tighter the gripper grips.
- `grasp_distance`: The grasping alignment distance, in the format `[desired distance in x`

`direction, desired distance in y direction, yaw angle]`, unit: meters/radians. When the chassis aligns with the target, it maintains the desired distance from the target.

- `place_distance`: The placement position reference and working range detection, in the format `[x reference, y reference, maximum mechanical distance on x axis, maximum mechanical distance on y axis]`, unit: meters. `[0]/[1]` are the reference coordinates for the chassis to align with the target, and `[2]/[3]` are used to determine whether the target point exceeds the reachable range of the robotic arm.
- `grasp_offset`: The grasp position offset, in the format `[x offset, y offset, z offset, grasp angle]`, with units of meters and degrees respectively, used to fine-tune the grasp coordinates.
- `place_offset`: The placement position offset, in the format `[x offset, y offset, z offset, placement pitch angle]`, with units of meters and degrees respectively, used to fine-tune the placement coordinates. The X/Y axis offsets follow the robot's right-hand coordinate system, and the Z axis is recommended to remain positive.
- `auto_back_dist`: The automatic retreat distance, unit: meters. After grasping/placement is complete, the chassis automatically retreats this distance to avoid collisions and entering the costmap restricted area.

7. Function Q&A

7.1 How to modify the distance between the car and the target point when placing an object?

- Use `AdjustChassisFitArmRange` to adjust the chassis position.

If the target point is outside the working range of the robotic arm, OpenClaw will call `AdjustChassisFitArmRange` to adjust the chassis position, so that the target point enters the reachable range of the robotic arm:

```
# Source code location: AdjustChassisFitArmRange()
def AdjustChassisFitArmRange(self, x: float, y: float, z: float) -> bool:
    delta_x = x - self.place_distance[0]  # Calculate the offset in the x direction
    delta_y = y - self.place_distance[1]  # Calculate the offset in the y direction
    self.move_by_odom(distance=abs(delta_x), direction='forward')
    direction = 'left' if delta_y > 0 else 'right'
    self.move_by_odom(distance=abs(delta_y), direction=direction)
    return True
```

Adjustment Method:

1. Modify the reference coordinates `[x reference, y reference]` in the `place_distance` parameter.
2. Increasing `place_distance[2]` and `place_distance[3]` can expand the working range of the robotic arm.

7.2 How to adjust when the placement point is inaccurate?

Check in sequence:

1. **Whether the placement point is within the maximum operating range of the robotic arm**
    - If it exceeds the actual operating range of the robotic arm, you can adjust `place_distance[0]` to reduce the alignment distance of the chassis before placement.
2. **Whether the placement offset is appropriate**
    - If the point is accurate but the placement is offset, adjust the `place_offset` parameter.
    - The X/Y axis offsets follow the robot's right-hand coordinate system, and the Z axis is recommended to remain positive.
3. **Whether the ambient lighting affects the depth camera**
    - Missing depth will cause inaccurate 3D coordinate conversion.
    - Improve the ambient lighting or adjust the position of the object.

- Parameter configuration example

```
mcp_service:
  ros__parameters:
    place_distance: [0.14, 0.0,0.25, 0.2]  # [x reference, y reference, maximum mechanical distance on x axis, maximum mechanical distance on y axis]
    place_offset: [-0.01,0.00,0.1,35.0]    # [x offset, y offset, z offset, placement pitch angle]
```